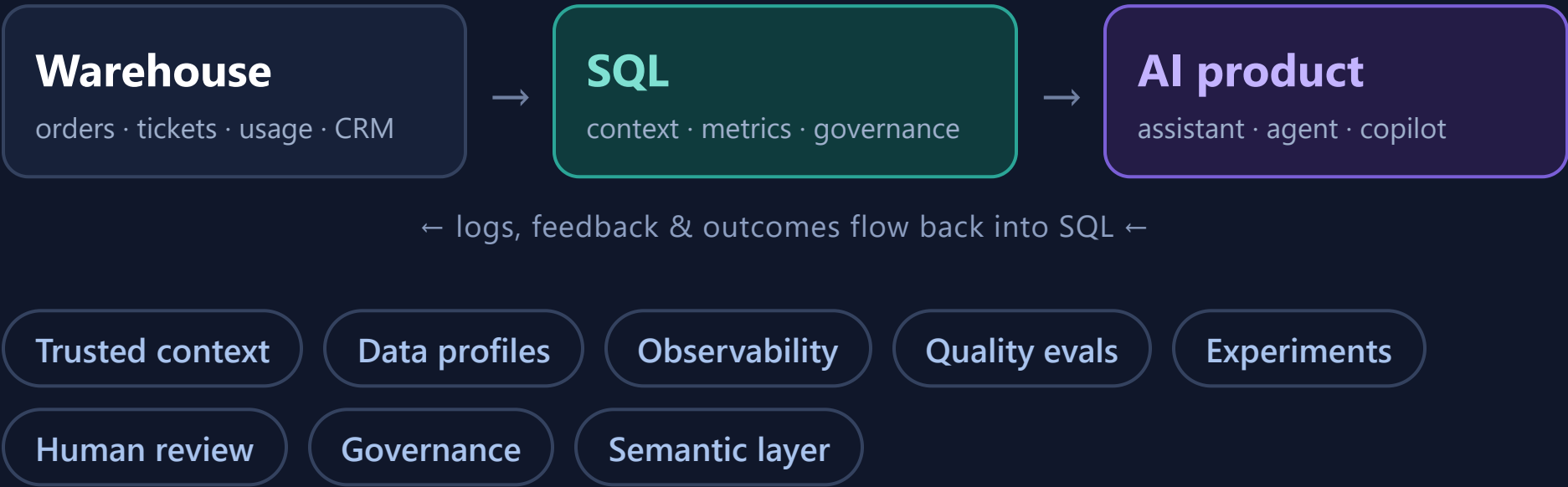


SQL’s New Job in the AI Era

How the language behind dashboards is powering reliable AI workflows.



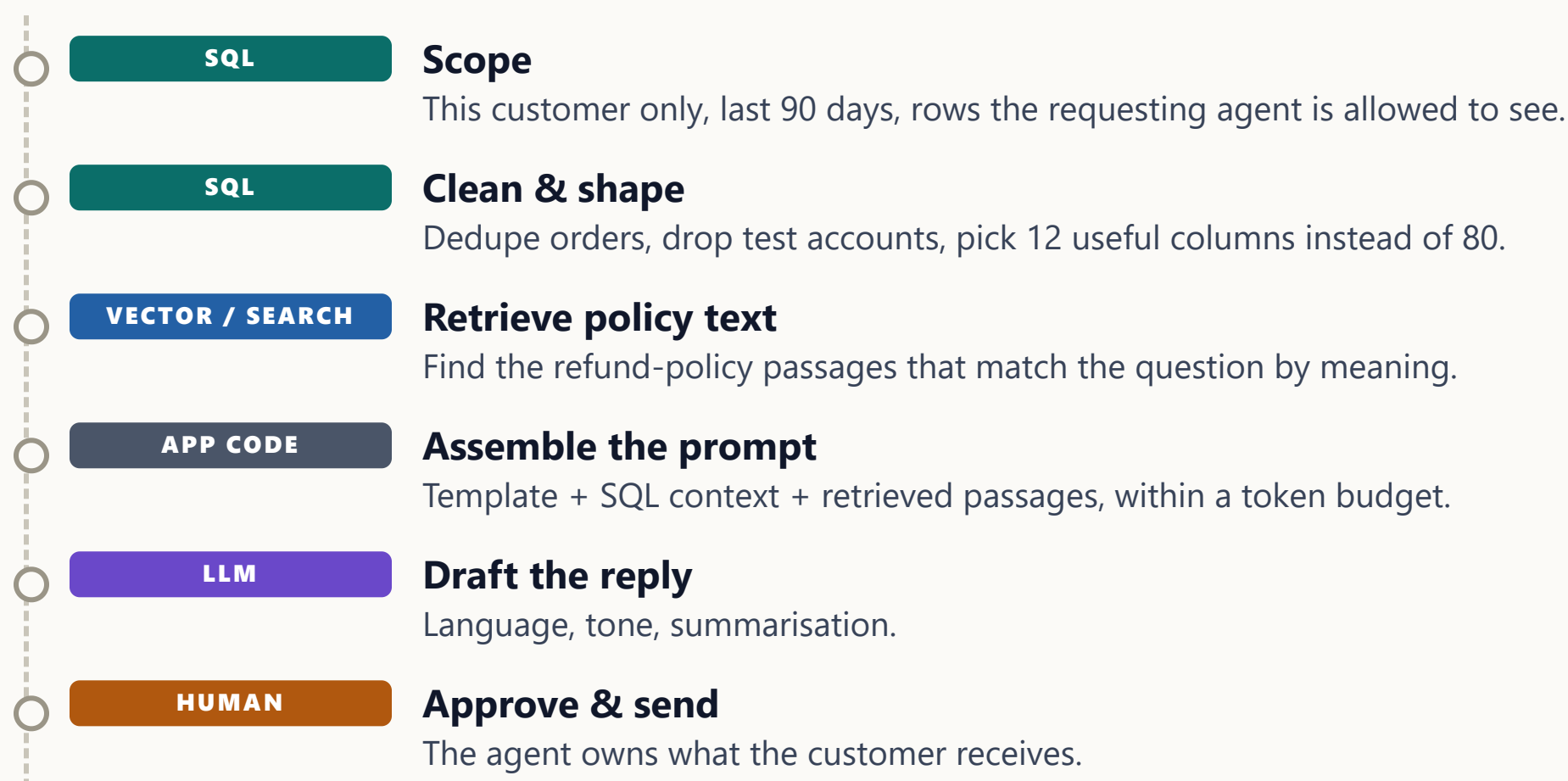
Sagar Rathkanthiwar

Data & AI Professional

10 slides · save for later →

Decide what the model sees — before it sees it

A support assistant drafting a reply for one customer doesn't need the warehouse. It needs **their** recent orders and open tickets — and nothing that belongs to anyone else.



SQL IS THE RIGHT TOOL FOR

Exact filters, joins across systems, freshness checks, permission scoping, reproducible inputs you can audit later.

SQL IS NOT THE TOOL FOR

Ranking free text by meaning, writing the reply, or deciding whether a reply is appropriate to send.

The cheapest hallucination fix is **better input**. Every wrong row in the context is a fact the model will confidently repeat.

Compress each customer into one row a model can read

SNIPPET 1 · COMPACT CUSTOMER CONTEXT

```
-- One compact, refreshable context record per customer
CREATE VIEW ai_customer_profile AS
SELECT
  c.customer_id,
  c.plan_tier,
  c.account_status,
  (SELECT SUM(o.amount) FROM orders o
   WHERE o.customer_id = c.customer_id) AS lifetime_value,
  (SELECT MAX(o.ordered_at) FROM orders o
   WHERE o.customer_id = c.customer_id) AS last_order_at,
  (SELECT COUNT(*) FROM product_events e
   WHERE e.customer_id = c.customer_id
        AND e.event_at >= CURRENT_DATE - 30) AS events_30d,
  (SELECT COUNT(*) FROM support_tickets t
   WHERE t.customer_id = c.customer_id
        AND t.status = 'open') AS open_tickets
FROM customers c;
```

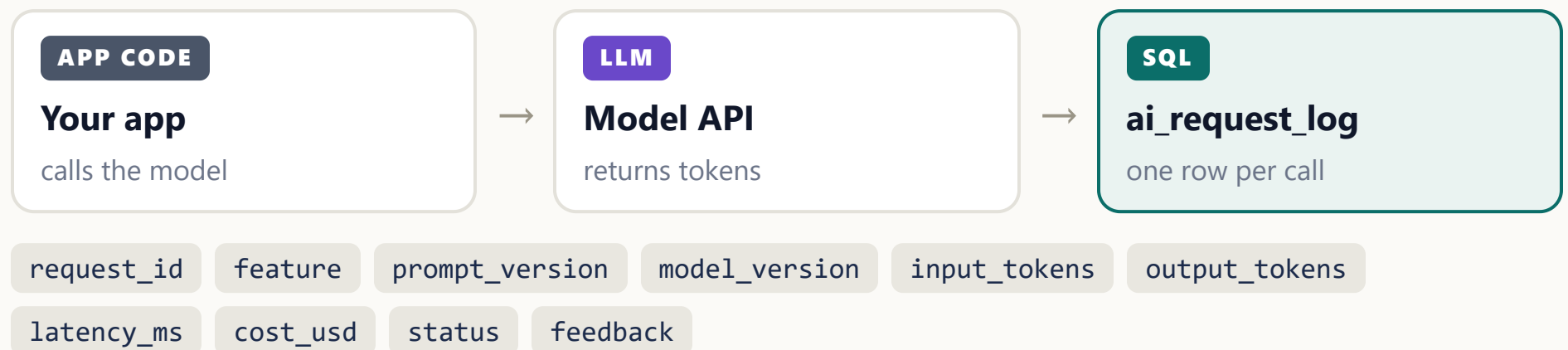
WHAT THE ASSISTANT RECEIVES

```
{ "plan_tier":      "Pro",
  "account_status": "active",
  "lifetime_value": 4820,
  "last_order_at":  "2026-09-18",
  "events_30d":     212,
  "open_tickets":   1 }
```

- **~60 tokens** instead of hundreds of raw order and event rows.
- **SQL does the arithmetic.** LLMs are unreliable at summing rows; they're good at explaining a number.
- **Materialise & refresh** on a schedule; add a `refreshed_at` stamp.

Compute the facts in SQL. **Let the model explain them — not invent them.**

Log every AI call. Query it like any fact table.



SNIPPET 2 · DAILY COST / QUALITY MONITOR

```

-- Daily cost, latency and quality by model + prompt version
SELECT
  CAST(created_at AS DATE)                AS day,
  model_version,
  prompt_version,
  COUNT(*)                                AS requests,
  SUM(cost_usd)                            AS cost_usd,
  AVG(latency_ms)                          AS avg_latency_ms,
  AVG(CASE WHEN status = 'error' THEN 1.0 ELSE 0 END) AS error_rate,
  AVG(CASE WHEN feedback = 'up' THEN 1.0
           WHEN feedback = 'down' THEN 0.0 END) AS thumbs_up_rate
FROM ai_request_log
WHERE created_at >= CURRENT_DATE - 14
GROUP BY CAST(created_at AS DATE), model_version, prompt_version
ORDER BY day DESC, cost_usd DESC;
  
```

Questions this answers:

Which prompt version doubled cost?

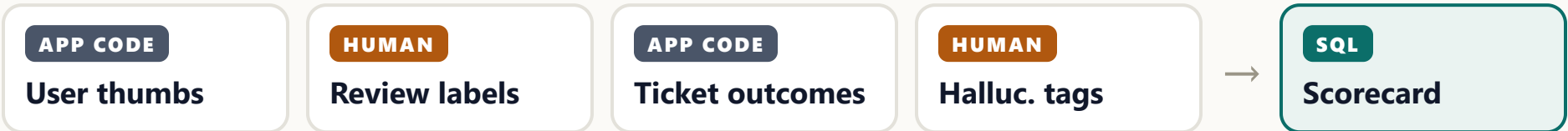
Did the new model slow responses?

Where are errors spiking?

Averages hide pain: add **p95 latency** with your warehouse's percentile function. And logged prompts can contain PII — **redact and set retention** like any customer data.

Turn scattered feedback into a quality scorecard

Join four signals on `request_id`: user thumbs, human review labels, ticket outcomes, and confirmed hallucination tags. Then slice by intent.



Support intent	Convos	Thumbs-up	Reviewed	Halluc. tag	Resolved, no escalation
Order status	4,210	88%	312	0.6%	81%
Product how-to	6,870	83%	402	1.2%	74%
Account access	980	70%	188	2.1%	55%
Billing dispute	1,145	61%	290	3.8%	42%

Illustrative numbers. The overall average (~80% thumbs-up) looks healthy — the billing segment is where the assistant is failing.

SQL DOES

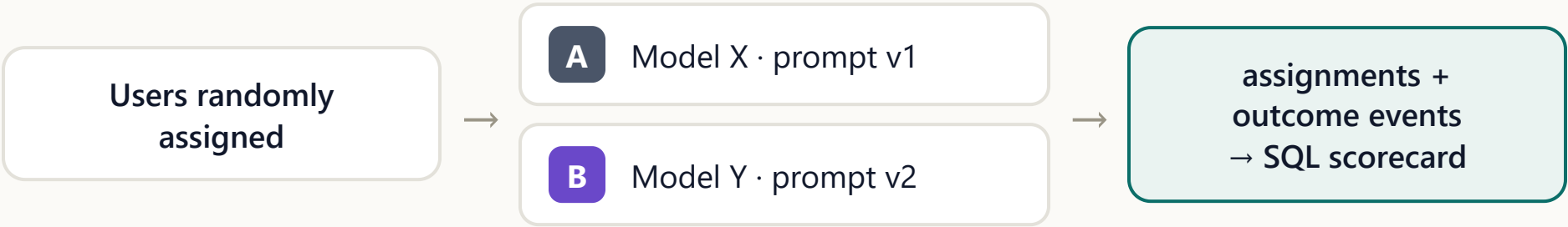
Aggregates, trends by version, segment slices, and **stratified samples** so reviewers see hard cases, not just easy ones.

SQL CAN'T

Judge whether an answer was actually correct. That takes human reviewers — or an LLM judge **calibrated against** them.

Thumbs are biased: few users vote, unhappy ones vote more. SQL won't tell you an answer was wrong — **it tells you where to look, and whether it's improving.**

Ship prompt v2 like a product change — not a vibe check



Sales-assistant metric	A · v1	B · v2	Change
Adoption (reps who used it)	34%	37%	+3 pts
Task completion	71%	76%	+5 pts
Satisfaction (1–5)	4.1	4.2	+0.1
Escalated to a human	18%	14%	–4 pts
Cost per completed task	\$0.042	\$0.061	+45%

Illustrative numbers.

- **Analyse by assigned variant**, not by who happened to use the feature.
- **Check the split** — a 50/50 test that lands at 56/44 means broken assignment.
- **SQL builds the inputs**; the significance test belongs in a stats library or notebook.
- **Segment the result** — new vs tenured reps, small vs large deals. An average win can hide a segment loss.
- **+5 pts completion for +45% cost** is a business trade-off. Humans decide it.

The winner isn't the variant that *feels* smarter. It's the one that **clears your quality bar at a cost you'll accept at scale.**

Use SQL to decide which drafts a person must see

SNIPPET 3 · ROUTE FOR HUMAN REVIEW

```
-- Route risky AI drafts to a human review queue
INSERT INTO review_queue (response_id, reason)
SELECT response_id, reason
FROM (
  SELECT
    r.response_id,
    CASE
      WHEN r.topic IN ('legal', 'refund_over_limit') THEN 'high_risk_topic'
      WHEN r.confidence < 0.70 THEN 'low_confidence'
      WHEN a.tier = 'enterprise'
        AND r.sentiment = 'negative' THEN 'key_account'
      WHEN r.cost_usd > 0.50 THEN 'expensive_request'
    END AS reason
  FROM ai_responses r
  JOIN accounts a ON a.account_id = r.account_id
  WHERE r.status = 'draft'
) flagged
WHERE reason IS NOT NULL;
```



- **“Confidence” is whatever your app produces** — a classifier score or a model self-rating. It is a signal, not a guarantee.
- **Calibrate thresholds with SQL:** how often did reviewers actually change a `low_confidence` draft?
- **Real-time routing often lives in app code.** SQL is where you design, backtest and audit the rules.

An AI tool should never get every column

customers column	AI assistant gets
customer_id	Yes
plan_tier	Yes
region	Row filter
email	Masked j***@acme.com
phone	Excluded
date_of_birth	Excluded
internal_notes	Excluded

- Least-privilege views**

Grant the AI service account a purpose-built view, never base tables.
- Row-level security**

The assistant inherits the *end user's* access — an EMEA agent sees EMEA customers.
- Masking**

Mask or tokenise PII at the column; don't hope the model ignores it.
- Auditability**

Log which identity queried what, when — and join it to the AI request log.

Same idea, different syntax: PostgreSQL row-level security · SQL Server RLS & dynamic data masking · Snowflake row access & masking policies · BigQuery authorized views & policy tags.

A prompt that says “don't reveal emails” is not access control. Instructions can be ignored or bypassed by prompt injection. Permissions enforced in the database can't be talked out of.

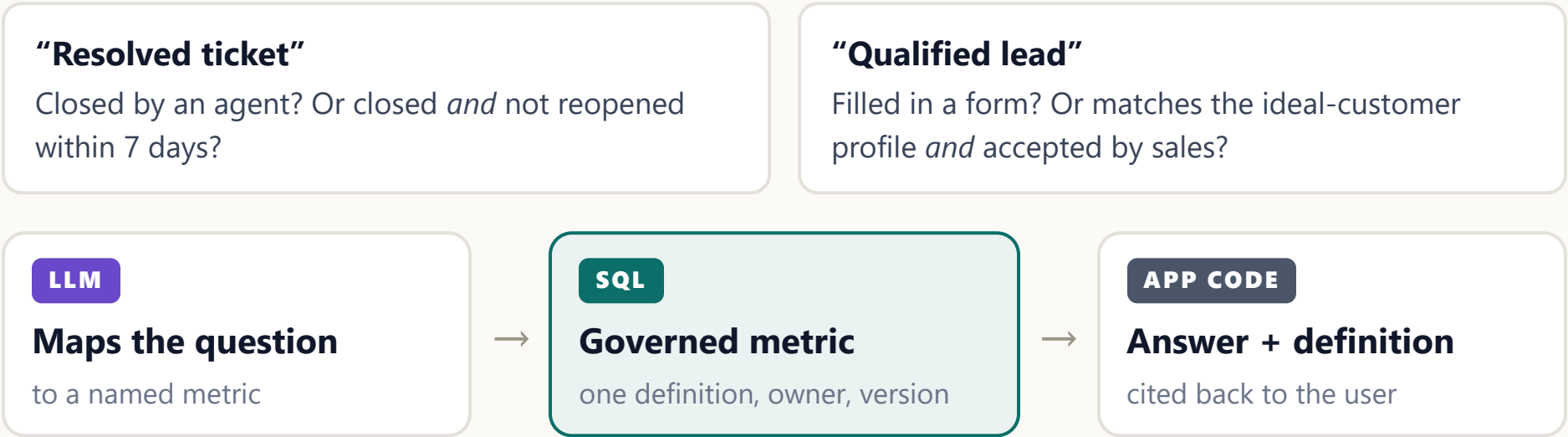
Rule of thumb: **if a column would be a problem in a screenshot, it's a problem in a prompt.**

One definition of “active” — before AI answers anything

“How many active customers do we have?” Three teams, three honest answers:

Team	“Active customer” means	Answer
Marketing	Logged in during the last 30 days	18,400
Finance	Paid an invoice in the last 90 days	11,250
Product	≥ 3 key actions in the last 28 days	7,900

Illustrative numbers. The same trap hides in every business noun:



- **Metric owners (humans)** agree the definition; the semantic layer or governed views encode it once.
- **The LLM translates language to metric names** — it should not re-derive the business logic on every question.

AI doesn’t resolve ambiguity in your metrics. **It automates it — at the speed of chat.**

THE TAKEAWAY

AI makes answers faster. SQL makes the inputs, measurements, and decisions trustworthy.

SQL

Select, measure, govern. Trusted context, cost and quality metrics, access rules, one metric definition.

VECTOR / SEARCH

Find relevant unstructured text — policies, docs, past tickets — by meaning.

LLM

Understand and generate language. Summarise, draft, classify, translate intent.

APP CODE

Orchestrate and enforce in real time. Prompts, retries, routing, guardrails.

HUMAN

Define, judge, decide. Metric definitions, quality calls, trade-offs, accountability.

Where could SQL make your AI workflow more reliable?

Share your most valuable SQL-for-AI use case in the comments — and save this for your next AI project review.

Sagar Rathkanthiwar · Data & AI Professional · follow for more field guides

Implementation note: table names are generic and examples use broadly portable SQL. Date arithmetic, percentile functions, row-level security, masking policies and semantic-layer tooling differ across PostgreSQL, SQL Server, Snowflake, BigQuery, Databricks and others — details vary by warehouse and product architecture.